New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

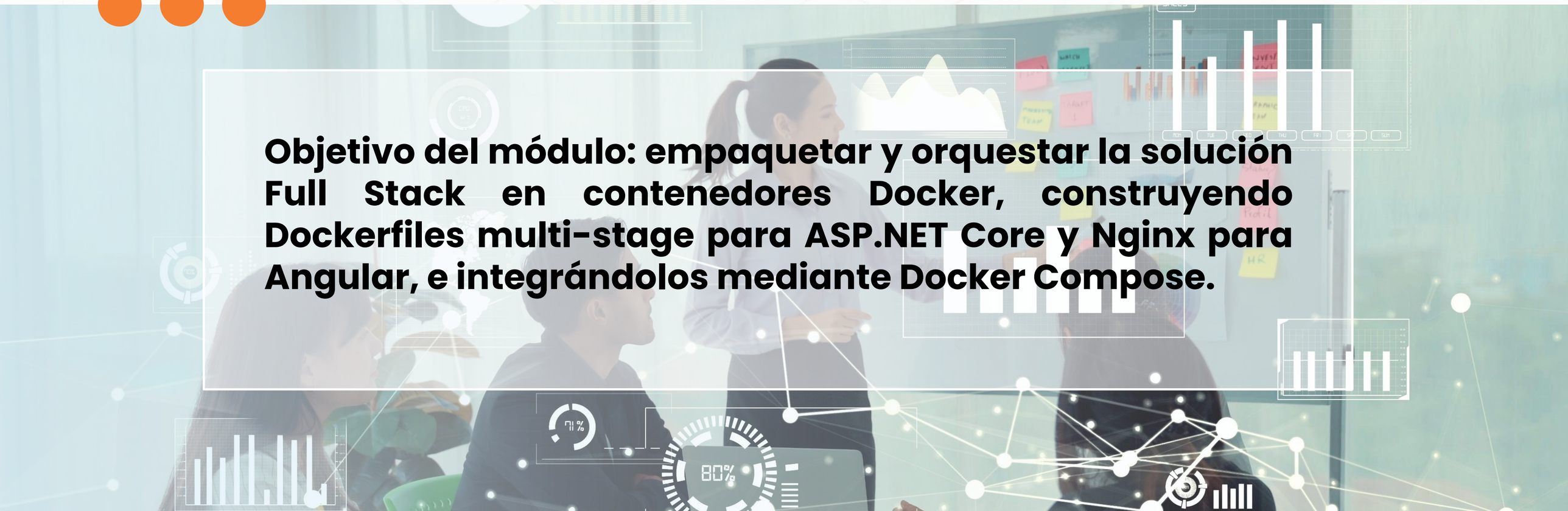
Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 18 — Módulo 4:

Contenedores con Docker y Docker Compose



Objetivo del módulo: empaquetar y orquestar la solución Full Stack en contenedores Docker, construyendo Dockerfiles multi-stage para ASP.NET Core y Nginx para Angular, e integrándolos mediante Docker Compose.



New Horizons®
Centro de Capacitación

Agenda

- Introducción a la contenedorización: Docker vs Máquinas Virtuales tradicionales
- Dockerfile multi-stage para ASP.NET Core 8 Web API: Capas build y runtime ligero
- Dockerfile para Angular 18: Build con Node LTS y servicio estático con Nginx Alpine
- Configuración de Nginx con redirección SPA para rutas de Angular (try_files)
- Orquestación completa con docker-compose.yml: Base de datos, API y Frontend



New Horizons®
Centro de Capacitación

Sesión 1:

Contenedores con Docker y Orquestación con Docker Compose

Visión general de la sesión

New Horizons®
Centro de Capacitación



Antes de los contenedores de barco, cada carga se estibaba a mano y se rompía en el camino. Docker hace con el software lo que el contenedor hizo con el comercio: una caja estándar que viaja igual en tu laptop, en QA y en la nube.



Portabilidad total

Adiós al “en mi máquina sí funciona”: la app viaja con su runtime y dependencias.

Piénsalo así: *La caja se carga igual en Callao, Rotterdam o Shanghái.*



Imágenes ligeras

Dockerfiles multi-stage: el SDK se queda en el taller y a producción solo va el resultado.

Piénsalo así: *Viajas con maleta de mano, no con la mudanza completa.*



Un solo comando

Docker Compose levanta base de datos, API y frontend con `docker compose up`.

Piénsalo así: *Un director de orquesta: una señal y todos tocan a tiempo.*

Pilares técnicos fundamentales

New Horizons
Centro de Capacitación

⚓ **Una naviera moderna en 5 piezas:** lo que necesitamos para llevar SolicitudesApp de tu laptop a cualquier puerto.

1 🏢 Contenedor vs VM

≈ Departamento vs casa

Comparte el kernel del host: arranca en segundos y pesa MB.

2 🏭 Multi-stage .NET 8

≈ Taller y vitrina

Se compila con el SDK; se ejecuta solo con el runtime ASP.NET.

3 🍞 Angular + Nginx

≈ La panadería

Node hornea el build; Nginx Alpine vende los estáticos.

4 🧭 try_files

≈ El recepcionista

Toda ruta desconocida vuelve a index.html y Angular decide.

5 🎼 docker-compose.yml

≈ El director de orquesta

BD, API y frontend con red, volúmenes y orden de arranque.

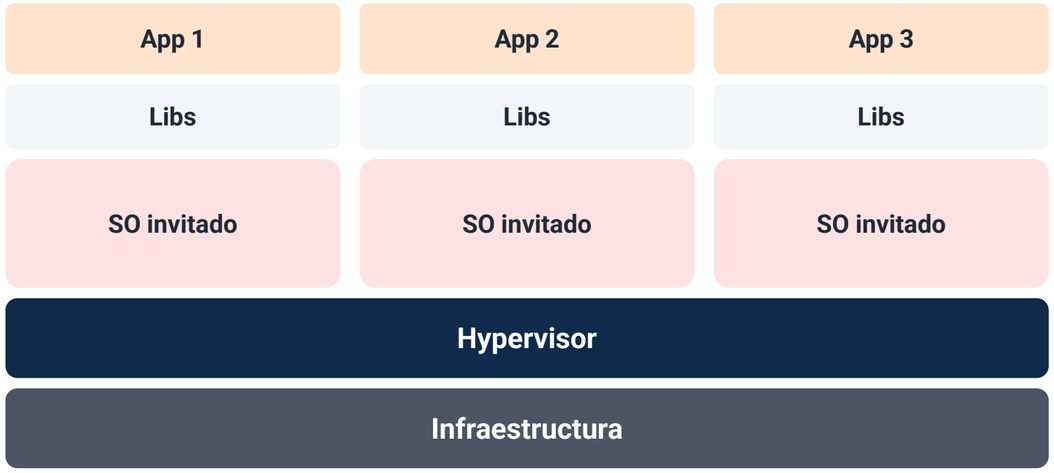
🧠 Cada metáfora volverá en el **resumen final**: si recuerdas la imagen, recuerdas el concepto.

Docker vs Máquinas Virtuales

New Horizons
Centro de Capacitación

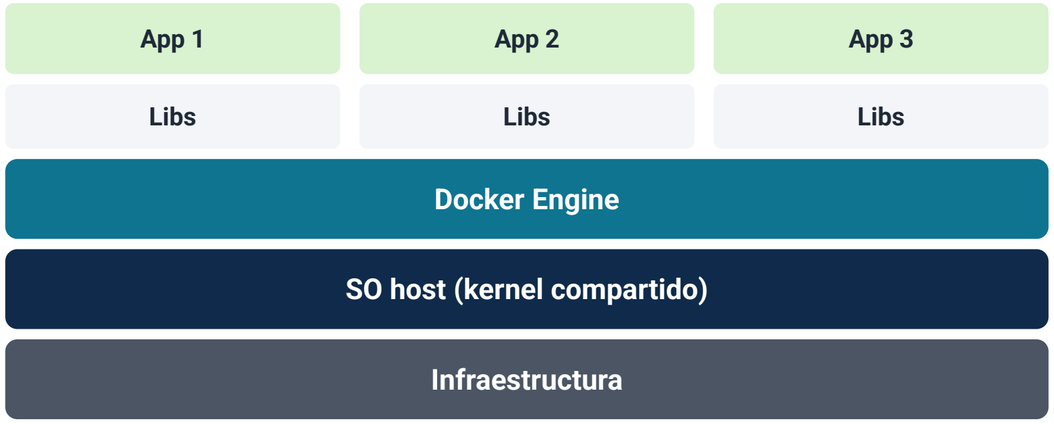
Casas vs departamentos: una VM es una casa independiente con sus propios cimientos (SO completo). Un contenedor es un departamento: comparte cimientos (el kernel del host) y solo trae sus muebles.

Máquinas virtuales



Arranque	Peso
minutos	**GB**

Contenedores



Arranque	Peso
segundos	**MB**

Práctica: Dockerización Multi-Stage de Backend y Frontend

New Horizons
Centro de Capacitación



Línea de montaje del astillero: primero fabricamos cada contenedor por separado, luego los subimos al mismo barco y probamos que zarpe.

1

Dockerfile multi-stage de la API

Lo escribimos en la raíz de SolicitudesApp (contexto de build = la solución).

```
$ docker build -t solicitudes-api .
```

2

Dockerfile de Angular + nginx.conf

Build con Node LTS y servicio estático con Nginx Alpine y try_files.

```
$ docker build -t solicitudes-web ./solicitudes-web
```

3

docker-compose.yml

Une base de datos, API y frontend en una red con volumen persistente.

```
$ docker compose up -d --build
```

4

Validación en limpio

Compilación desde cero de todos los componentes y prueba de punta a punta.

```
$ docker compose ps  →  http://localhost:8080
```

Anatomía de un Dockerfile: la receta

New Horizons
Centro de Capacitación

Un Dockerfile es una receta de cocina: cada instrucción es un paso y la imagen es el plato listo para servir.

FROM
La base: elegir la olla

Imagen de partida (sdk, aspnet, node, nginx).

WORKDIR
Tu mesa de trabajo

Carpeta donde se ejecutan los siguientes pasos.

COPY
Traer los ingredientes

Copia archivos del contexto a la imagen.

RUN
Cocinar

Ejecuta comandos al construir (restore, publish, npm ci).

EXPOSE
La ventana de atención

Documenta el puerto que escucha la app.

ENTRYPOINT
Servir el plato

Comando que corre al iniciar el contenedor.

Cada paso = una capa

```
ENTRYPOINT
COPY --from=build
FROM aspnet:8.0
RUN dotnet publish
COPY . .
RUN dotnet restore
FROM sdk:8.0
```

Las capas se apilan y se **reutilizan desde caché** si nada cambió debajo de ellas.

Dockerfile multi-stage para ASP.NET Core 8

New Horizons
Centro de Capacitación

Dockerfile (raíz de SolicitudesApp)

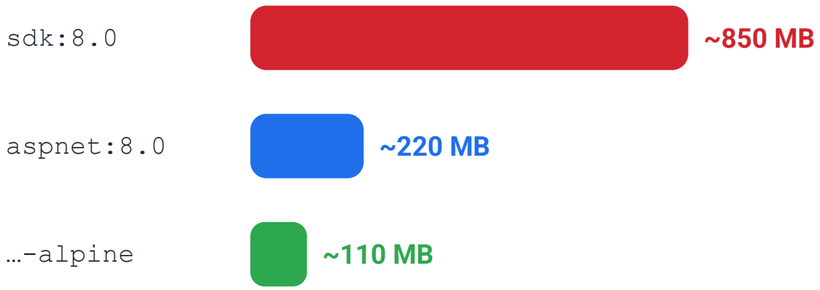
```
# Etapa 1: build con el SDK completo
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY SolicitudesApp.Api/*.csproj SolicitudesApp.Api/
RUN dotnet restore SolicitudesApp.Api
COPY . .
RUN dotnet publish SolicitudesApp.Api -c Release \
    -o /app/publish /p:UseAppHost=false

# Etapa 2: runtime ligero (solo ASP.NET Core)
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "SolicitudesApp.Api.dll"]
```

El taller y la vitrina

El taller (SDK) tiene todas las herramientas; a la vitrina (runtime) solo llega el producto terminado.

Peso de la imagen base (aprox.)



2 Primero el .csproj, luego restore

Si no cambian los paquetes, la capa viene de caché.

4 COPY --from=build

Solo cruza lo publicado; el SDK se queda atrás.

Capas y caché: el orden importa

New Horizons
Centro de Capacitación

🧳 **Arma la maleta como un viajero experto:** lo que casi nunca cambia va al fondo; lo que cambias a diario, arriba. Así no desarmas todo por un detalle.

❌ Orden ingenuo

`COPY . .`	cambia con cada línea de código
`RUN dotnet restore`	se repite SIEMPRE ⏱ ~60 s
`RUN dotnet publish`	se repite

✅ Orden inteligente

`COPY *.csproj`	cambia solo al agregar paquetes
`RUN dotnet restore`	CACHED ⚡ 0 s
`COPY . .`	cambia con tu código
`RUN dotnet publish`	se repite (inevitable)

`.dockerignore`

```
bin/  obj/  node_modules/
.git/  *.user  .env
```

🚫 **.dockerignore** es la lista de “no empacar”: contexto más chico, builds más rápidos y ningún .env ni secreto filtrado dentro de la imagen.

Dockerfile para Angular 18 con Nginx Alpine

New Horizons
Centro de Capacitación

solicitudes-web/Dockerfile

```
# Etapa 1: build con Node LTS
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build -- --configuration production

# Etapa 2: servir estáticos con Nginx
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/solicitudes-web/browser \
     /usr/share/nginx/html
EXPOSE 80
```



La panadería

El horno (Node) se queda en la cocina; al mostrador (Nginx) solo sale el pan horneado: HTML, JS y CSS.

1 **node:20-alpine**

Node LTS para compilar; no viaja a producción.

2 **npm ci**

Instala exactamente lo del package-lock: builds reproducibles.

3 **nginx:1.27-alpine**

Servidor web de ~20 MB que entrega archivos estáticos.

4 **dist/.../browser**

Angular 17+ deja el build en la subcarpeta browser.

nginx.conf: redirección SPA con try_files

solicitudes-web/nginx.conf

```
server {
  listen 80;
  server_name _;
  root /usr/share/nginx/html;
  index index.html;

  # Rutas de Angular: si no es un archivo, va al index
  location / {
    try_files $uri $uri/ /index.html;
  }

  # Peticiones a la API: proxy al contenedor "api"
  location /api/ {
    proxy_pass http://api:8080;
  }
}
```

1

2

El recepcionista del edificio

Si la oficina que pides no existe como archivo, te manda al lobby (index.html) y Angular Router te guía.

Tres visitas a Nginx

✅ `/assets/logo.png`

El archivo existe → se sirve

✅ `/solicitudes/15`

No existe → index.html → Router

⛔ `F5 sin try_files`

Nginx busca el archivo → 404

💡 ② Con el proxy, Angular llama a `/api/...` relativo: sin CORS ni URL fija.

docker-compose.yml: la orquesta completa

New Horizons
Centro de Capacitación

docker-compose.yml

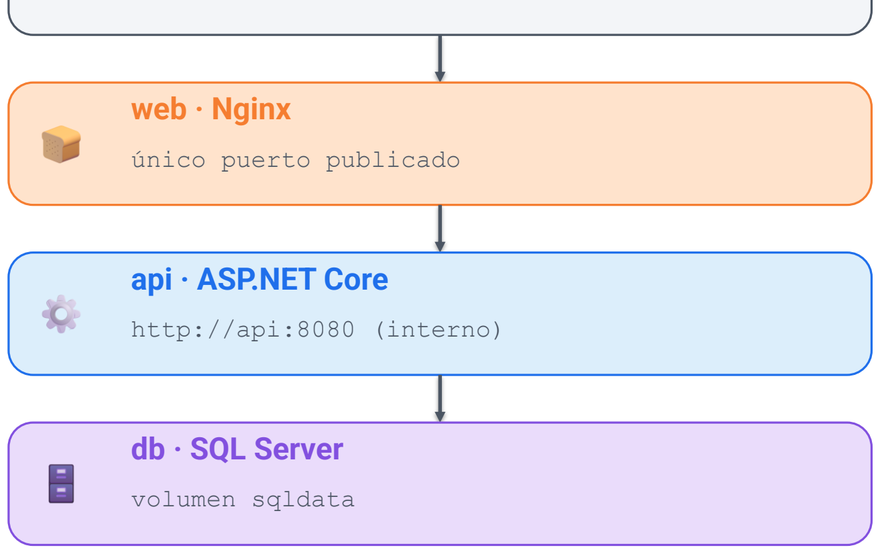
```yaml
services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: ${DB_PASSWORD}
    volumes:
      - sqldata:/var/opt/mssql
  api:
    build: .
    env_file: .env
    depends_on: [db]
  web:
    build: ./solicitudes-web
    ports:
      - "8080:80"
    depends_on: [api]

volumes:
  sqldata:
```

1. Servicio `db`: imagen oficial de SQL Server 2022.
2. Volumen `sqldata` para persistir los datos.
3. Servicio `api` construido desde el Dockerfile raíz.
4. Servicio `web` publicado en el puerto 8080.

El director de orquesta

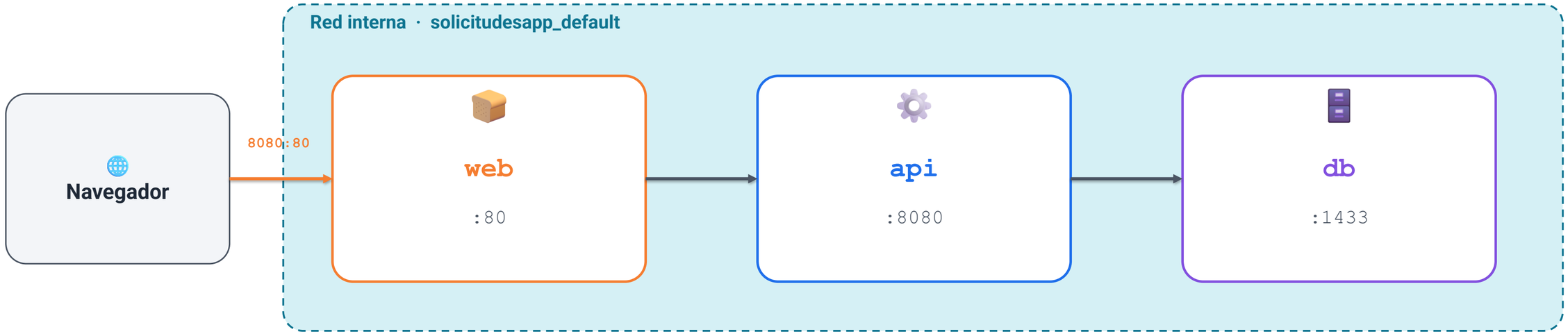
Cada servicio es un músico; Compose les da la partitura, el escenario (red) y la señal de entrada.



Redes internas: cómo se encuentran los contenedores

New Horizons
Centro de Capacitación

☎ **Como los anexos de una oficina:** dentro, marcas “api” o “db” y te contestan; desde la calle solo existe una línea pública (el puerto 8080).



Navegador → web	`http://localhost:8080`
web → api	`http://api:8080`
api → db	`Server=db,1433`

⚠ **Trampa clásica:** `localhost` dentro de un contenedor es el propio contenedor. Para llegar a la BD usa el nombre del servicio: `db`.

Volúmenes: datos que sobreviven

New Horizons
Centro de Capacitación

terminal

```
# Apaga y elimina contenedores; los datos quedan a salvo
docker compose down

# Vuelve a levantar: SQL Server monta el mismo volumen
docker compose up -d

# ⚠ Elimina contenedores Y volúmenes: datos perdidos
docker compose down -v

# Inspeccionar los volúmenes
docker volume ls
docker volume inspect solicitudesapp_sqldata
```

1
2
3

Habitación de hotel y depósito

El contenedor es la habitación: al salir, se limpia. El volumen es el depósito del hotel: tus cosas siguen ahí.

¿Qué pasa con las solicitudes guardadas?

up

Se crean 12 solicitudes

down → up

Siguen las 12 solicitudes

down -v → up

Base de datos vacía

El nombre real del volumen lleva como prefijo el nombre del proyecto.

Variables de entorno y secretos

New Horizons
Centro de Capacitación

.env + docker-compose.yml

```
# .env  (no se versiona: agrégalo a .gitignore)
DB_PASSWORD=Demo-NoUsar-EnProd-2026!
JWT_KEY=CLAVE-DEMO-NO-USAR-EN-PROD-000000000000

# docker-compose.yml  (servicio api)
  api:
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      Jwt__Key: ${JWT_KEY}
      ConnectionStrings__Default: >-
        Server=db;Database=Solicitudes;User Id=sa;
        Password=${DB_PASSWORD};TrustServerCertificate=True
```



Misma receta, distinta sazón

La imagen es idéntica en DEV y PROD; lo que cambia es la configuración que le inyectas al arrancar.

1. **${VAR} sale del .env**

   Compose lo reemplaza al levantar; el secreto no entra a la imagen ni a Git.

2. **Doble guion bajo __ = :**

   `Jwt__Key` sobrescribe `Jwt:Key` de appsettings.json.

3. **Angular no necesita variables**

   Con el proxy de Nginx usa `/api` relativo: la misma imagen sirve en todos los ambientes.

Comandos esenciales del día a día

New Horizons
Centro de Capacitación

Tu control remoto de Docker: seis botones resuelven el 90% del trabajo diario con Compose.

Play

```
docker compose up -d --build
```

Construye si hace falta y levanta todo en segundo plano.

Stop

```
docker compose down
```

Detiene y elimina contenedores (no los volúmenes).

Lista de asistencia

```
docker compose ps
```

Qué servicios corren, su estado y sus puertos.

La caja negra

```
docker compose logs -f api
```

Sigue en vivo lo que imprime la API.

Entrar a la habitación

```
docker compose exec api sh
```

Abre una terminal dentro del contenedor.

Limpieza general

```
docker system prune
```

Libera espacio de imágenes y capas sin usar.

Flujo completo: de compose up al navegador

New Horizons
Centro de Capacitación

Abrir el restaurante cada mañana: llegan los insumos, se prende la cocina, se abre el salón… y recién entra el primer cliente.

1

Lee la partitura

Compose interpreta docker-compose.yml y el .env

2

Construye

Imágenes de api y web (o las toma de caché)

3

Prepara el local

Crea la red interna y el volumen sqldata

4

Arranca en orden

db → api → web, según depends_on

5

Primer cliente

El navegador abre localhost:8080

El viaje de una petición: “Aprobar solicitud 15”

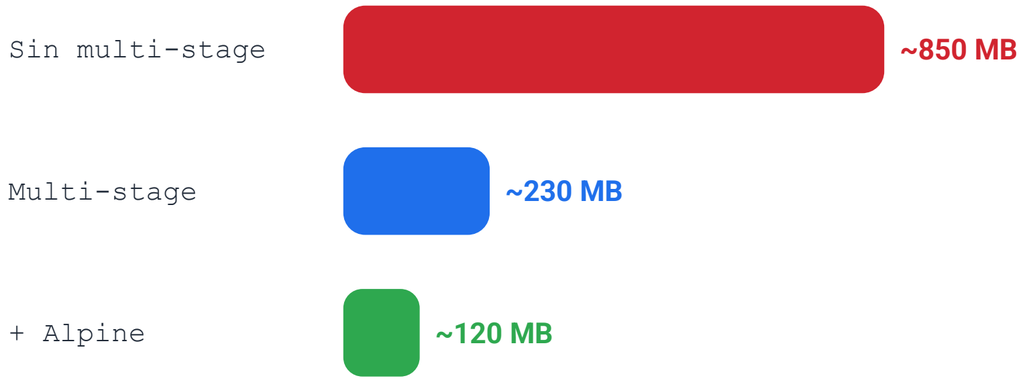
Navegador → Nginx (web) → /api → api:8080 → SQL Server (db) → 200 OK ✓

Imágenes ligeras y seguras

New Horizons
Centro de Capacitación

🎒 **Viaja con maleta de mano:** menos peso es menos tiempo de descarga, menos superficie de ataque y despliegues más rápidos.

Imagen final de la API (aprox.)



📏 Mídelo tú mismo: `docker images` muestra el tamaño real de cada imagen construida.

⛰️ Bases Alpine o slim

aspnet:8.0-alpine, node:20-alpine, nginx:alpine.

🚫 .dockerignore

Fuera bin/, obj/, node_modules/, .git y .env.

👤 Sin root

Las imágenes .NET 8 traen el usuario `app`: agrega `USER app`.

📌 Versiones fijas

nginx:1.27-alpine en vez de latest: builds reproducibles.

❤️ Healthcheck

Que Docker sepa si la API está viva, no solo encendida.

Diagnóstico: cuando algo falla

New Horizons
Centro de Capacitación

El manual del mecánico: síntoma → causa probable → reparación. Casi todo se detecta con docker compose logs.

Síntoma	Causa probable	Solución
404 al refrescar con F5	Falta try_files en nginx.conf	`try_files $uri $uri/ /index.html`
La API no conecta a la BD	Usa localhost o la BD aún no está lista	`Server=db + reintentos/healthcheck`
Se perdieron los datos	Se ejecutó down -v o no hay volumen	`Declarar volumen sqldata`
Puerto ya está en uso	Otro proceso usa el 8080 del host	`Cambiar a "8081:80"`
Mis cambios no aparecen	Se reutilizó la imagen anterior	`docker compose up -d --build`

Errores comunes y cómo evitarlos

New Horizons®
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Omitir la validación estricta de parámetros en el servidor	Validar siempre los contratos DTO en cada llamada HTTP
Almacenar información sensible en repositorios públicos	Utilizar variables de entorno seguras en desarrollo y producción
Copiar el .env o appsettings con claves dentro de la imagen	Inyectar secretos al arrancar y listarlos en .dockerignore
Conectar a la BD con localhost desde el contenedor	Usar el nombre del servicio: Server=db
Usar la etiqueta latest en producción	Fijar versiones: nginx:1.27-alpine, aspnet:8.0

Checklist antes de zarpar

New Horizons®
Centro de Capacitación



Inspección antes de zarpar: ningún capitán sale del puerto sin revisar la carga; tú no despliegas sin esta lista.

- ✓ **Dockerfiles multi-stage para API y frontend** → *Imágenes pequeñas, sin SDK ni Node en producción.*
- ✓ `.dockerignore` **con bin/, obj/, node_modules/, .env** → *Contexto liviano y sin secretos filtrados.*
- ✓ `try_files` **configurado en nginx.conf** → *El F5 en cualquier ruta de Angular funciona.*
- ✓ **Volumen declarado para SQL Server** → *Los datos sobreviven a un down / up.*
- ✓ **Secretos en .env (fuera de Git) o gestor de secretos** → *Nada sensible en la imagen ni en el repositorio.*
- ✓ **Build en limpio:** `docker compose up -d --build` → *Lo que funciona aquí funciona en el servidor.*

Resumen de la Sesión 18

New Horizons®
Centro de Capacitación

📦 Contenedores

Portabilidad total: adiós al “en mi máquina sí funciona”.

≈ El contenedor de barco

🏭 Multi-stage .NET

Compilar con el SDK y ejecutar solo con el runtime.

≈ El taller y la vitrina

📦 Angular + Nginx

Nginx Alpine sirve la SPA, ultraligero y rápido.

≈ La panadería

🧭 try_files

Toda ruta de Angular vuelve a index.html.

≈ El recepcionista

🎼 Docker Compose

Todo el ecosistema con docker compose up.

≈ El director de orquesta

🔐 Redes y volúmenes

BD aislada en la red interna y datos persistentes.

≈ Anexos y depósito del hotel

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Por qué los Dockerfiles multi-stage reducen el tamaño final de la imagen en más del 80%?
2. ¿Por qué es indispensable configurar try_files en Nginx para aplicaciones Angular SPA?
3. ¿Cómo se comunican los contenedores de frontend, backend y base de datos dentro de Docker Compose?
4. ¿Cómo persistimos los datos de PostgreSQL o SQL Server al reiniciar los contenedores?
5. ¿Qué variables de entorno debemos inyectar en tiempo de ejecución en cada contenedor?

New Horizons
Centro de Capacitación

Conclusiones



Portabilidad

Docker garantiza portabilidad absoluta eliminando el clásico 'en mi máquina sí funciona'.



Frontend ultraligero

Nginx Alpine ofrece un servidor web ultraligero y de alto rendimiento para servir la SPA.



Un solo comando

Docker Compose permite levantar todo el ecosistema con un único comando docker compose up.



Base de datos aislada

Las redes internas de Docker aíslan la base de datos protegiéndola del acceso público directo.

Lista para la nube

La aplicación queda lista para ser desplegada en cualquier proveedor Cloud o clúster Kubernetes.

GRACIAS